پیش گزارش آزمایش ۳ شبکه

آشا عسکری 40331053

1) پروتکل HTTP یک پروتکل در لایه کاربرد (Application layer) است که برای انتقال داده‌ها در وب استفاده می‌شود. این پروتکل براساس مدل درخواست - پاسخ (Request - Response) بین کلاینت و سرور کار می‌کند. مراحل عملکرد HTTP:

1) برقراری اتصال:

کلاینت برای برقراری ارتباط، ابتدا یک اتصال TCP با سرور برقرار می‌کند Three-way Handshake. در نسخه‌ی امن یعنی HTTPS، قبل از تبادل داده، فرایند TLS Hand Shake نیز انجام می‌شود. (port 80 برای HTTP و port 443 برای HTTPS معمولاً)

2) ارسال درخواست (HTTP Request) ، پیام درخواست HTTP دارای سه بخش اصلی است:

- Request line : شامل متد (DELETE ، PUT ، POST ، GET ، ...) - آدرس منبع (URL) و نسخه پروتکل (مثل HTTP/1.1)
- Header lines ، شامل اطلاعات متادیتا مانند نوع محتوا (content_Type) ، طول محتوا (content_length) ، کوکی‌ها و ...
- entity Body : در صورت نیاز (مثلاً در post)

3) پردازش در سرور: درخواست توسط سرور دریافت می‌شود و آن را پردازش می‌کند (مثلاً خواندن فایل یا اجرای برنامه سمت سرور)

4) ارسال پاسخ (Response):

سرور یک پاسخ HTTP را برمی‌گرداند که شامل: - کد وضعیت (مثل 200 موفق ، 404 پیدا نشد) هدرها و بدنه پاسخ (مثل Json و HTML ، تصویر و ...)

5) پایان یا ادامه اتصال: در HTTP/1.0 اتصال معمولاً بسته می‌شود اما در HTTP/1.1 امکان اتصال پایدار وجود دارد.

ویژگی‌ها: 1) Stateless : هر درخواست مستقل است و سرور اطلاعاتی از درخواست‌های قبلی نگه نمی‌دارد.

2) ساده و قابل توسعه است - طوری که با استفاده از Header ها قابل گسترش است.

3) متنی بودن: پیام‌ها به صورت متنی قابل خواندن هستند.

4) نسخه امن HTTP یعنی HTTPS که با استفاده از رمزنگاری (SSL / TLS) امنیت ارتباط را فراهم می‌کند.

Scanned with CamScanner

2) پروتکل HTTps، نسخه امن HTTp است که با استفاده از TLS ارتباط بین کلاینت و سرور را رمزنگاری می‌کند. TLS از ترکیب رمزنگاری نامتقارن (Asymmetric) و متقارن (Symmetric) برای تامین امنیت استفاده می‌کند.

- مراحل عملیات رمزنگاری (TLS Handshake):

1) Client Hello: کلاینت لیستی از الگوریتم‌های رمزنگاری (Cipher Suites)، نسخه‌ی TLS و یک عدد تصادفی (Client Random) را برای سرور ارسال می‌کند.

2) Server Hello: سرور یک Cipher Suite مناسب را انتخاب کرده و همراه با گواهی دیجیتال (digital certificate) خود و یک عدد تصادفی دیگر (Server Random) ارسال می‌کند.

3) احراز هویت سرور: کلاینت گواهی سرور را با استفاده از کلید عمومی مرجع صادرکننده (CA) بررسی می‌کند تا از معتبر بودن سرور مطمئن شود.

4) تبادل کلید (Key-Exchange):

- در روش‌های قدیمی: کلاینت یک Pre-Master-Secret را با کلید عمومی سرور رمز کرده و ارسال می‌کند. اما در روش‌های جدید (مثل Diffie-Hellman): کلاینت و سرور بدون ارسال مستقیم کلید، یک کلید مشترک تولید می‌کنند.

5) تولید کلیدهای جلسه (Session Keys):

با استفاده از Server Random، Client Random و Pre-Master-Secret، کلیدهای متقارن برای رمزنگاری داده‌ها تولید می‌شود.

6) شروع ارتباط امن:

از این مرحله به بعد، داده‌ها با الگوریتم‌های متقارن (مثل AES) رمز و ارسال می‌شوند.

نامتقارن: برای تبادل امن کلید (امن ولی کند)

متقارن: برای انتقال داده‌ها ([illegible] و کارآمد)

ویژگی‌ها: 1) محرمانگی: جلوگیری از خواندن داده‌ها توسط افراد غیرمجاز

2) یکپارچگی: با استفاده از MAC یا AEAD تضمین می‌شود داده تغییر نکرده است.

3) احراز هویت: معمولاً سرور (و در موارد خاص کلاینت) تایید می‌شود.

3) ارتباط بین کلاینت و سرور در HTTp ابتدا با تبدیل نام دامنه به IP توسط DNS شروع می‌شود. سپس کلاینت یک اتصال TCP (به پورت 80 برای HTTp معمولی یا 443 برای HTTPs) ایجاد می‌کند. این اتصال با انجام فرایند

NAHAL

Scanned with CamScanner

فرایند سه مرحله‌ای معروف به Three_way handshaking برقرار می‌شود: کلاینت یک بسته‌ی SYN می‌فرستد، سرور با SYN_ACK پاسخ می‌دهد و نهایتاً کلاینت با ارسال ACK اتصال را تایید می‌کند. پس از برقراری اتصال مطمئن TCP، کلاینت یک پیام درخواست HTTP شامل خط درخواست (متد، مسیر منبع، نسخه HTTP)، Header‌ها مانند (Host و User-agent) و در صورت نیاز بدنه (مثلاً در متد POST) را برای سرور ارسال می‌کند. سرور درخواست را دریافت، پردازش و سپس یک پیام پاسخ HTTP شامل خط وضعیت (نسخه HTTP)، کد وضعیت مثل (200 OK)، هدرهای پاسخ مانند (content_type) و بدنه حاوی محتوای درخواست شده (HTML، JSON و غیره) را از طریق اتصال TCP به کلاینت باز می‌گرداند. در نسخه HTTP/1.0 معمولاً پس از هر درخواست و پاسخ اتصال بسته می‌شود، اما در HTTP/1.1 به صورت پیش‌فرض اتصال پایدار نگه داشته می‌شود (persistant) تا چندین درخواست و پاسخ متوالی از یک اتصال TCP استفاده کنند. در HTTP/2 این اتصال پایدار به صورت چندگانه‌سازی شده و چند درخواست همزمان را امکان‌پذیر می‌کند، در HTTP/3 به جای TCP از پروتکل QUIC روی UDP استفاده می‌شود که اتصال را سریع‌تر برقرار می‌کند.

4) چه تفاوت‌هایی بین HTTP/1.1 و HTTP/2 وجود دارد؟ در HTTP/1.1 درخواست‌ها یکی پس از دیگری فرستاده می‌شوند و هر درخواست باید پاسخ بگیرد تا بعدی برود، این موضوع باعث کندی می‌شود، اما در HTTP/2 چندین درخواست و پاسخ همزمان روی یک اتصال TCP رفت و آمد می‌کنند. فرمت داده در HTTP/1.1 متنی است اما در HTTP/2 به فرمت دودویی (باینری) است که پردازش آن سریع‌تر است. Header‌ها در HTTP/1.1 تکراری و حجیم هستند اما در HTTP/2 فشرده می‌شوند. HTTP/2 قابلیت سرور پوش را دارد یعنی سرور می‌تواند قبل از درخواست کلاینت، منابع مورد نیاز را بفرستد، اما این HTTP/1.1 این قابلیت را ندارد. در نهایت HTTP/1.1 برای سرعت مجبور است چندین اتصال TCP موازی باز کند در حالی که HTTP/2 فقط با یک اتصال TCP همه کارها را انجام می‌دهد و بار کمتری روی سرور می‌گذارد.